SMART TRAFFIC MANAGEMENT SYSTEM

A Multi-Agent AI System with Max-Pressure Signal Control, A* Routing, Emergency Preemption, Negotiated Bus Priority and Hill-Climbing Policy Tuning

AI Case Study Report | Review 1 and Review 2

Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore

#	Team member	Role
M1	P.M. Radha Krishna | CB.SC.U4CSE23134	Environment & Simulation
M2	Member 2 | (Name, Roll No.)	Intersection Agent & Signal Control
M3	Member 3 | (Name, Roll No.)	Vehicle Agent & Route Search
M4	Member 4 | (Name, Roll No.)	Priority Vehicles
M5	Member 5 | (Name, Roll No.)	Coordinator, Protocols & Testing

Batch 2027, Section B

Abstract

Fixed-time traffic signals cannot react to real demand, they make ambulances wait at red lights, and they give drivers no warning of accidents. This project models a city grid as a **multi-agent system** of five cooperating agent types. **Intersection agents** switch signals with the decentralised **Max-Pressure** algorithm plus an anti-starvation term. **Vehicle agents** plan routes with **A* search** over their own beliefs about the road network. **Ambulance agents** preempt signals ahead of them. **Bus agents** negotiate priority, and intersections may refuse. A **Coordinator (Traffic Management Centre)** broadcasts incidents and tunes every controller with **hill climbing**. All interaction goes through FIPA-ACL messages.

Every vehicle is simulated to completion and results are averaged over several random seeds. Compared with a 20-second fixed-time plan, Max-Pressure cuts average waiting time by **47.5%** (15.45 s to 8.11 s). Ambulance preemption makes the ambulance **29.6%** faster with zero red-light delay. A* finds the optimal route every time while expanding **14.1** nodes against **36** for Uniform-Cost Search. Hill climbing lowers waiting time on unseen traffic by a further **20%**. The system scales to an 8x8 grid with 461 agents. All 43 automated tests pass. Negative results are reported alongside the positive ones.

1. Problem Statement and Objectives

Urban intersections are the bottleneck of city traffic. Three failures of conventional control motivate this work:

- **Static timing.** A fixed cycle gives the same green to an empty side street as to a rush-hour arterial.
- **Emergency delay.** An ambulance crossing a signalised corridor loses tens of seconds at red lights.
- **Information lag.** Drivers learn about an accident only when they reach the queue behind it.

Objectives. Build a working multi-agent system that (1) adapts signals to live queues without a central controller, (2) guarantees that no approach is starved, (3) gives emergency vehicles a green corridor safely, (4) lets public transport request priority without overriding everyone else, (5) spreads incident information so vehicles re-plan early, and (6) improves its own control parameters. Each objective is backed by a measured, repeatable test.

1.1 Team and work division

The system is split so that every member owns one agent or algorithm end to end: code, scenario, tests, slides and viva questions.

#	Member	Owns	Code	Evaluation
M1	P.M. Radha Krishna CB.SC.U4CSE23134	Environment & Simulation	road_network.py, traffic_grid.py, simulator.py, demo.py	S8 Scalability, route demo
M2	Member 2 (Name, Roll No.)	Intersection Agent & Signal Control	intersection_agent.py, max_pressure.py	S1, S2
M3	Member 3 (Name, Roll No.)	Vehicle Agent & Route Search	vehicle_agent.py, search_astar.py	S4, S7, A* trace
M4	Member 4 (Name, Roll No.)	Priority Vehicles	emergency_agent.py, bus_agent.py	S3, S5
M5	Member 5 (Name, Roll No.)	Coordinator, Protocols & Testing	coordinator_agent.py, message.py, hill_climbing.py, tests/	S6, test suite

2. Review 1: PEAS Formulation

PEAS (Performance, Environment, Actuators, Sensors) is written for each of the five agent types. The performance measure is what the agent's decision procedure actually optimises in the code, not just a wish list.

Agent	Performance measure	Environment	Actuators	Sensors
Intersection (M2)	Min. average vehicle delay; max. throughput; no starvation (bounded red while cars wait); safe clearance (3 s yellow)	4-leg junction, 2 incoming + 2 outgoing roads per axis, stochastic arrivals, neighbouring junctions	Signal phase: NS green / EW green / yellow; messages AGREE, ACCEPT/REJECT_PROPOSAL, INFORM	Queue detectors on all 8 roads (loop counts); message inbox (preemption, bus proposals, platoons, parameter updates)
Vehicle (M3)	Min. travel time to destination; min. stops; never enter a closed road	Directed road graph, signals, other cars ahead, incidents	Accelerate / stop, cross stop line, choose next road (route)	Own position & speed; signal state at stop line; gap to car ahead; TMC broadcasts; sees blocked road at junction
Ambulance (M4)	Min. response time; zero red-light delay; no unsafe signal switch	Same roads with background traffic	Drive at 60 km/h; PREEMPT_OVERRIDE, CANCEL	Route look-ahead (next 2 junctions, ETA); AGREE replies
Bus (M4)	Min. passenger delay (40 passengers) without excessive harm to cross traffic	Mixed traffic, junctions that may refuse	Drive at 40 km/h; PROPOSE, CANCEL	Next junction ETA; ACCEPT/REJECT_PROPOSAL replies
Coordinator (M5)	Network-wide average delay; incident awareness latency; delay on unseen traffic after tuning	Whole network (aggregate densities), incident reports	BROADCAST INCIDENT_ALERT, INFORM CONGESTION_WARNING, INFORM PARAMETER_UPDATE	Density of every road; incident reports; simulated evaluation of candidate parameters

3. Review 1: Environment and Agent Analysis

3.1 Task-environment properties

Property	Classification	Justification in this system
Observable	Partially	An intersection sees only its own 8 roads. A vehicle sees the signal and the car ahead. Incidents elsewhere are known only through messages (vehicles keep explicit known_blocked / known_speed beliefs).
Agents	Multi-agent: cooperative and competitive	Intersections cooperate (green-wave hand-off). Vehicles compete for road space and green time. The bus competes with cross traffic, and the intersection arbitrates.
Deterministic	Stochastic	Poisson arrivals, random origin-destination pairs, unplanned incidents. Seeded for reproducibility.
Episodic	Sequential	Holding a green now changes every queue later. Routing choices commit a vehicle to a road.
Static	Dynamic	Queues grow while agents deliberate. A decision must be made every 1 s tick.
Discrete	Continuous (discretised)	Positions and speeds are continuous. Time is discretised to 1 s, and routes to a graph of junctions.
Known	Known physics, unknown demand	Agents know road lengths, speed limits and the BPR model, but not future arrivals or incidents.

3.2 Agent architectures

Agent	Architecture (Russell & Norvig)	Why
Intersection	Utility-based agent with internal state	Keeps timers (phase duration, red time per axis) and pending requests, and picks the phase with the highest pressure score (its utility).
Vehicle	Goal-based, model-based agent	Has a goal (destination) and a model (beliefs about road costs), and uses search (A*) to find an action sequence. Re-plans when its model changes.
Ambulance	Goal-based agent with communication	Same goal-based core, plus a proactive protocol that changes other agents' behaviour ahead of time.
Bus	Utility-based negotiating agent	Proposes priority. The outcome depends on the intersection's evaluation of cross-street cost.
Coordinator	Learning agent	Performance element = the intersections' controllers; critic = simulated average delay; learning element = hill climbing; problem generator = random restarts.

3.3 Architecture of the multi-agent system

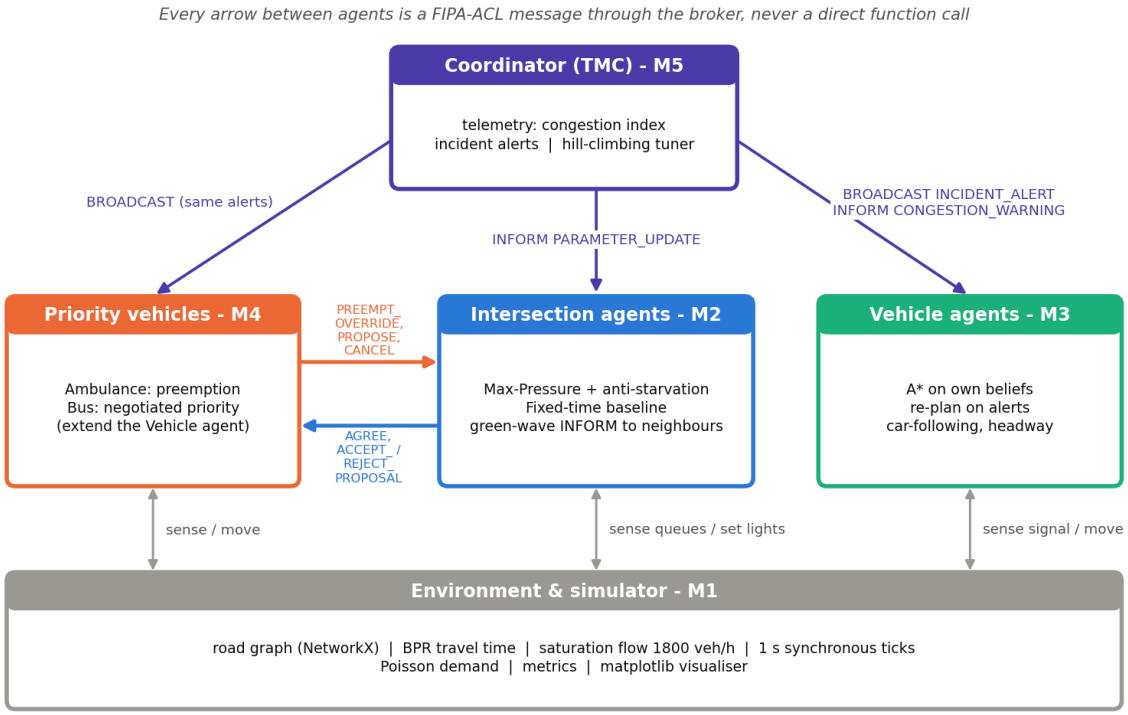


Figure 1. Agents, owners and the messages that connect them

4. Review 1: Algorithmic Modelling and Search Strategy

Four algorithms are at the centre of the system. The first, A*, is the core search strategy and is covered in the most depth.

4.1 Route search as a state-space problem (Vehicle agent, M3)

Component	Definition
State	The intersection node the vehicle is at, s in V
Initial state	Origin node (or the end of the road it is currently driving on, when re-planning)
Actions	Drive along any outgoing road u -> v that the vehicle believes is open
Transition model	RESULT(u, drive(u->v)) = v
Goal test	s == destination
Step cost	c(u,v) = BPR travel time = t0 (1 + 0.15 (V/C)^4), where t0 = length / believed speed
Heuristic	h(n) = straight-line distance(n, goal) / v_max, with v_max = the highest speed limit in the network (13.88 m/s)

A* algorithm

```
A*(start, goal):
    frontier <- priority queue ordered by f(n) = g(n) + h(n);  push(start, f = h(start))
    g[start] <- 0;  closed <- {}
    while frontier not empty:
        n <- pop lowest f
        if n in closed: continue            # stale entry
        if n == goal: return path(n)        # optimal: h is consistent
        closed.add(n)
        for each road n -> m believed open:
            g' <- g[n] + c(n, m)            # BPR congested travel time
            if g' < g[m]:  g[m] <- g';  parent[m] <- n;  push(m, g' + h(m))
    return failure                           # goal unreachable
```

Why the heuristic is admissible and consistent

- **Admissible (h(n) <= h*(n)).** Any route from n to the goal is at least the straight-line distance long, and no road is faster than v_max. So the true remaining time is at least d(n, goal) / v_max = h(n). Congestion only increases travel time above free flow, so admissibility holds under any load. Test UT-01 checks this for every node.
- **Consistent (h(u) <= c(u,v) + h(v)).** Triangle inequality: d(u,g) <= d(u,v) + d(v,g). Also, c(u,v) >= length(u,v) / v_max >= d(u,v) / v_max. Dividing by v_max gives the inequality. So f never decreases along a path, the first time the goal is popped it is optimal, and no node is re-expanded. Test UT-02 checks every edge.
- **Manhattan variant.** On this grid every road is axis-aligned, so Manhattan distance / v_max is also admissible, and tighter. It is used only as an experiment, because on a general map it could over-estimate.

Worked example: A* step by step

A congested 3x3 grid with road INT_10->INT_11 closed; trip W0 -> E2. Each row is one expansion. Notice that f(n) never decreases, and nodes with a high h (N0, W1) are never expanded.

```
A* trace on a congested 3x3 grid, W0 -> E2   (v_max = 13.88 m/s, h = Euclidean/v_max)
Blocked road(s): INT_10->INT_11

step  expand      g(n)    h(n)    f(n)   frontier (f, node) after pop
   1  W0           0.0   64.44   64.44
   2  INT_00     14.41   51.95   66.37
   3  INT_01     28.82   40.76   69.58   (74.5,INT_10), (90.3,N0)
   4  INT_10     28.96   45.57   74.52   (75.5,INT_02), (77.2,INT_11), (90.3,N0), (95.2,N1)
   5  INT_02     43.23   32.22   75.45   (77.2,INT_11), (87.5,INT_20), (90.3,N0), (95.2,N1), (102.8,W1)
   6  INT_11     44.99   32.22   77.21   (78.0,INT_12), (86.8,E0), (87.5,INT_20), (90.3,N0), (95.2,N1) ...
   7  INT_12     57.64   20.38   78.02   (86.8,E0), (87.5,INT_20), (89.1,INT_21), (90.3,N0), (95.2,N1) ...
   8  INT_22     72.05   14.41   86.46   (86.5,E1), (86.8,E0), (87.5,INT_20), (89.1,INT_21), (90.3,N0) ...
   9  E2         86.46    0.0    86.46   (86.5,E1), (86.8,E0), (87.5,INT_20), (89.1,INT_21), (90.3,N0) ...

A*  path: W0 -> INT_00 -> INT_01 -> INT_02 -> INT_12 -> INT_22 -> E2
    cost = 86.46 s, nodes expanded = 9
UCS path: W0 -> INT_00 -> INT_01 -> INT_02 -> INT_12 -> INT_22 -> E2
    cost = 86.46 s, nodes expanded = 20
Same optimal cost; A* expands fewer nodes because h(n) steers it toward the goal.
```

Comparison of search strategies (Scenario S7)

Measured on a 6x6 (60 nodes, 168 roads) with random congestion and one closed road, over 200 random trips. Optimality is checked against Uniform-Cost Search.

Algorithm	Nodes expanded	Max frontier	Optimal routes	Avg extra cost	Complete / optimal?
BFS	35.5	9.9	56.5%	0.67%	yes / only for equal costs
UCS (Dijkstra)	36.0	11.4	100%	0%	yes / yes
Greedy Best-First	7.7	10.7	51.5%	0.7%	yes (finite graph) / no
A* (Euclidean h)	14.1	12.3	100%	0%	yes / yes
A* (Manhattan h)	11.3	11.9	100%	0%	yes / yes (grid only)
Weighted A* (w=2)	8.0	10.4	57%	0.51%	yes / within factor 2

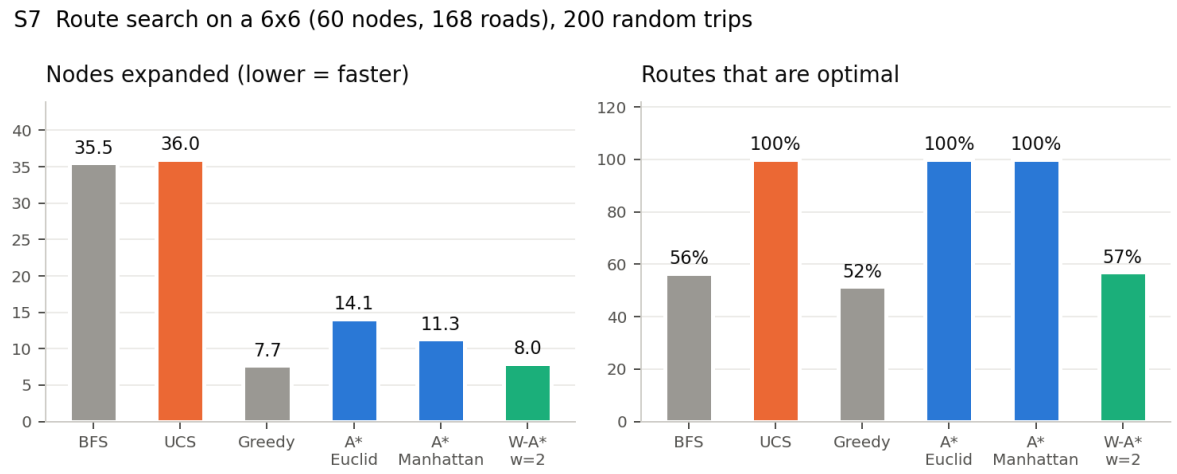


Figure 2. A is the only strategy that is both optimal and cheap*

Take-away. A* expands 61% fewer nodes than UCS and never loses optimality. Greedy and Weighted A* are cheaper still, but choose a worse route in about half of the trips.

4.2 Max-Pressure signal control (Intersection agent, M2)

Each intersection has two phases: North-South green and East-West green. A **movement** l -> m carries traffic from incoming road l to outgoing road m. Max-Pressure (Varaiya, 2013) serves the phase whose movements have the largest queue differential. It needs only local queues, and it is proven to stabilise every demand that any controller can stabilise.

```
w(l, m)  = Q_l - Q_m                                  movement pressure
P(p)     = sum over movements of p of max(0, Q_l - Q_m)  phase pressure
S(p)     = P(p) + beta * T_red(p)                        + anti-starvation (beta = 0.5 /s)

every second, after min_green (5 s):
    if the red approach has no waiting car:  keep current green
    elif phase duration >= max_green (30 s):  switch (via 3 s YELLOW)
    elif S(red) > S(green) + theta (1.0):      switch (via 3 s YELLOW)
    else keep current green
```

Worked example (INT_00, NS currently green for 12 s)

Movement	Q_in	Q_out	max(0, Q_in - Q_out)
N0 -> INT_00 -> INT_10 (southbound)	8	2	6
INT_10 -> INT_00 -> N0 (northbound)	3	5	0
W0 -> INT_00 -> INT_01 (eastbound)	2	1	1
INT_01 -> INT_00 -> W0 (westbound)	4	0	4

P(NS) = 6 + 0 = 6 and P(EW) = 1 + 4 = 5, so pure Max-Pressure would keep NS green. With anti-starvation, S(NS) = 6 + 0.5 x 0 = 6 and S(EW) = 5 + 0.5 x 12 = **11**. Since 11 > 6 + 1, the agent starts a yellow and then serves East-West. Test UT-18 reproduces this arithmetic.

Why the starvation term matters (S2). Without it, a lone side-street car can wait forever. The heavy axis always has more pressure, and 1 car vs 0 cars never beats the switching threshold. In the rush-hour test, **14 of 108** East-West cars were never served without the term, and **0** with it.

Green-wave hand-off (peer cooperation)

When a new green releases 4 or more queued vehicles, the intersection sends INFORM PLATOON_INCOMING to its neighbour on the same axis. For 25 s the neighbour adds 1.0 x (expected vehicles) to that axis' pressure, **but only to extend a green it is already showing**. It never forces a switch, because an earlier version that did made things worse.

4.3 Hill climbing policy tuning (Coordinator agent, M5)

Component	Definition
State	x = (min_green, max_green, beta, theta) on a discrete grid of 7 x 7 x 7 x 6 = 2,058 settings
Neighbours	Change one parameter by one grid step up or down (at most 8 neighbours)
Objective J(x)	Average vehicle delay in a simulation of asymmetric rush-hour demand (3 training seeds); lower is better
Move rule	Steepest descent: go to the best neighbour if it improves J; otherwise stop at a local optimum
Escaping local optima	2 random restarts; keep the global best. Results are memoised, so revisited states cost nothing.
Deployment	Best parameters are sent to every intersection with INFORM PARAMETER_UPDATE (test

Component	Definition
	IT-06)

```
HILL-CLIMB(start):
    for s in [start] + random_states(restarts):
        current <- s
        loop:
            best <- argmin over n in NEIGHBOURS(current) of J(n)
            if J(best) >= J(current): break            # local optimum
            current <- best
        keep the best local optimum seen
```

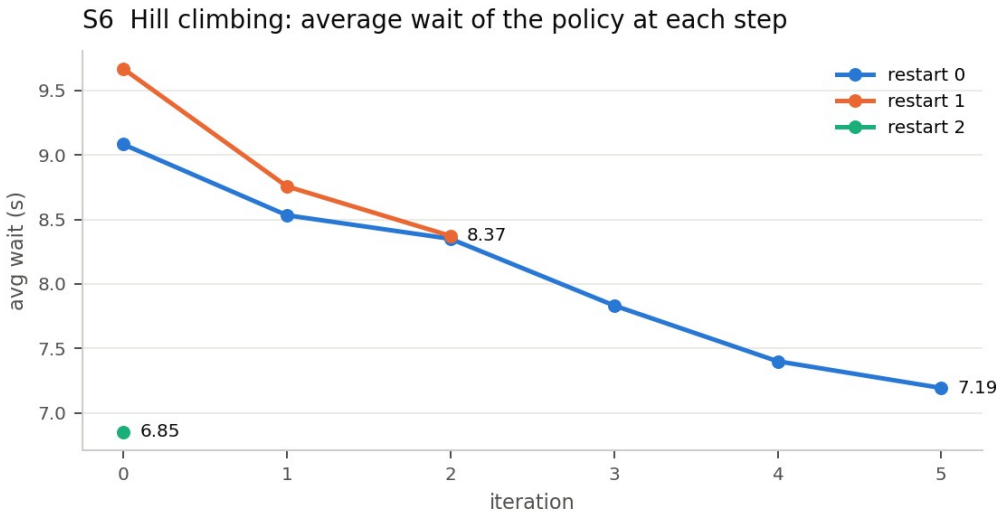


Figure 3. Hill-climbing trace: restart 1 stops at a worse local optimum; restarts matter

4.4 Interaction protocols (Priority vehicles, M4)

- **Emergency preemption (fipa-request).** When the ETA to a junction is 15 s or less, the ambulance sends PREEMPT_OVERRIDE with its axis. The intersection replies AGREE, and if the conflicting phase is green it goes through the full 3 s yellow before giving green. It holds the green until the ambulance sends CANCEL after crossing (safety timeout 40 s).
- **Transit Signal Priority (fipa-propose).** When its ETA is 12 s or less, the bus sends PROPOSE. The intersection accepts only if the cross-street queue is 6 or less and no ambulance is active. It then extends or advances the bus' green by at most 15 s, never cutting the minimum green. Otherwise it replies REJECT_PROPOSAL with a reason.

5. Review 2: Tool and Package Selection and Setup

Tool	Used for	Why this and not an alternative
Python 3.10+	All agents, algorithms, simulator	Readable enough for a 5-person team; standard in AI courses; dataclasses and typing keep interfaces clear
NetworkX	Road graph storage (DiGraph)	Standard graph library. The search algorithms are written by hand (not nx.astar_path) so each one can be explained and instrumented.
rich	Console tables for the scenarios	Readable output during the review without building a GUI
HTML + JavaScript canvas	The one-page route demo (demo.html)	Opens in any browser, offline, nothing to install or serve
matplotlib	Report charts	Available everywhere
unittest	43 automated tests with IDs	Part of the standard library, so there is nothing extra to install on lab machines
Not used: SUMO, Mesa, JADE	-	SUMO would hide the vehicle model we want to explain; Mesa/JADE add framework code without adding AI content. We implemented the message broker ourselves (about 150 lines) so every protocol step is visible.

Setup and run

```
pip install -r requirements.txt          # networkx, rich, matplotlib
python run_simulation.py --all           # full demo (about 1 minute)
python run_demo.py                       # THE DEMO: opens demo.html (original route vs
re-route)
python run_simulation.py --scenario 3    # one scenario: 1, 2 ... 8
python run_simulation.py --astar-trace   # A* expansion table
python run_simulation.py --protocol      # live message log
python tests/run_tests.py                # 43 tests -> results/test_report.json
python generate_results.py               # regenerate every number and chart
```

6. Review 2: Multi-Agent Execution and Interaction

6.1 Execution cycle

The simulator is synchronous and deterministic for a given seed (test IT-09). Each 1-second tick runs: (1) the Coordinator (telemetry, advisories), (2) every Intersection agent, (3) every Vehicle, Ambulance and Bus agent, (4) metrics. Every agent runs the same cycle in `BaseAgent.step()`: **receive messages -> perceive -> reason -> act**, and `reason()` runs exactly once per tick. The original code called it twice, which made signal timers run at double speed; this was fixed and is now covered by test IT-02.

6.2 Message catalogue

Performative	From -> To	Content	Effect
PREEMPT_OVERRIDE	Ambulance -> Intersection	axis, ETA	Intersection clears the conflicting phase (via yellow) and holds green
AGREE	Intersection -> Ambulance	status, axis	Confirms the corridor
PROPOSE	Bus -> Intersection	axis, ETA, passengers	Request for bounded priority
ACCEPT_ / REJECT_PROPOSAL	Intersection -> Bus	reason	Negotiation outcome
CANCEL	Ambulance/Bus -> Intersection	cleared	Release priority; Max-Pressure resumes
INFORM (green-wave)	Intersection -> neighbour	axis, vehicles	Neighbour extends green for the arriving platoon
BROADCAST INCIDENT_ALERT	Coordinator -> all	road, BLOCKED / SLOWED / CLEARED	Vehicles update beliefs and re-plan with A*
INFORM CONGESTION_WARNING	Coordinator -> all	index, hotspots	Vehicles re-plan if they find a route at least 15% faster
INFORM PARAMETER_UPDATE	Coordinator -> Intersections	tuned parameters	Controllers adopt the hill-climbing result

6.3 A real interaction from the demo run

In the demo run an ambulance, a bus and an accident overlap, and the agents resolve the conflict themselves. The bus' proposal is **rejected** while the ambulance holds INT_10. The bus had already avoided the slowed road INT_10->INT_11 because of the TMC broadcast, and its later proposals are accepted.

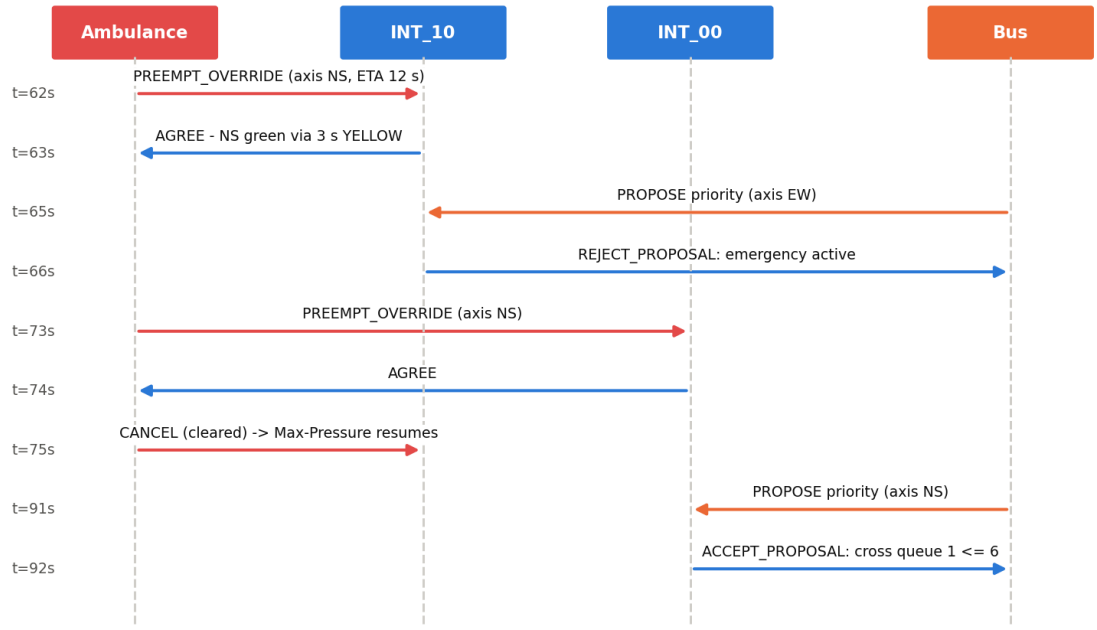


Figure 4. Message sequence recorded from the demo (results/protocol_trace.txt)

The demo page (python run_demo.py) shows one thing clearly: the original route of a car and how it adjusts. Six cars start on their cheapest A* route. At t = 10 s an accident closes road J01-J11 and the traffic centre broadcasts INCIDENT_ALERT. Only the cars that had that road on their route (Car 1 and Car 5) run A* again; the other four carry on unchanged.

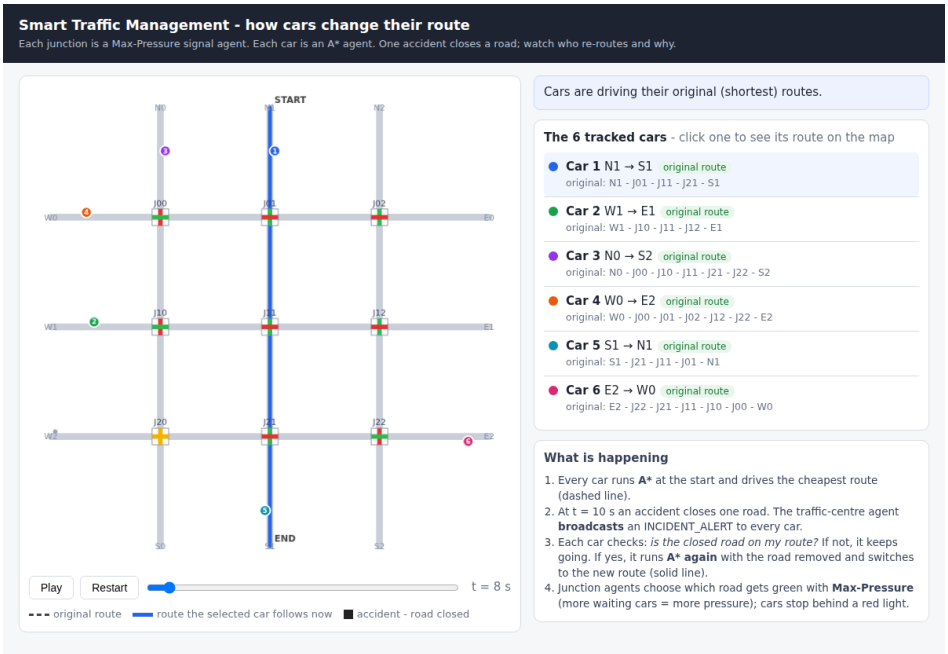


Figure 5a. Before the accident: Car 1 follows its original route (dashed) straight down column 1

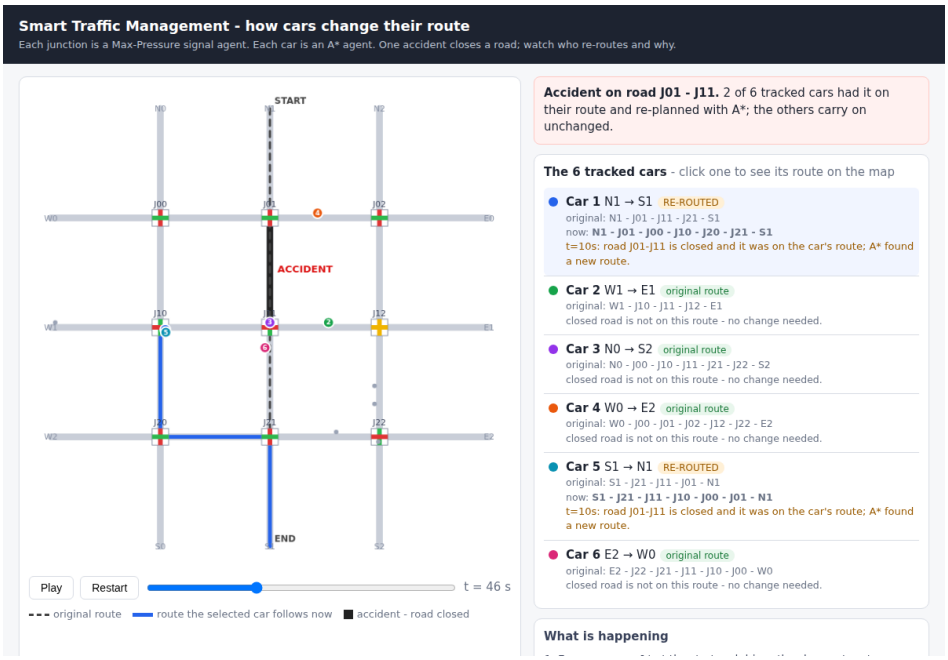


Figure 5b. After the accident: Car 1 has switched to a new route (solid) around the closed road; the panel states why

7. Review 2: Demo Quality and Testing Scenarios

Method. Vehicles arrive as Poisson processes with random origin-destination pairs. Every run continues **until the last vehicle arrives**, so no slow vehicle is silently dropped from the averages. Controller comparisons use identical demand (same seed) and are averaged over 5 to 10 seeds. Standard deviations are shown where relevant. Every number below comes from `results/results.json` and can be regenerated with `python generate_results.py`.

S1. Fixed-Time vs Max-Pressure (2x2 grid, 10 seeds)

Controller	Avg wait (s)	95th pct wait (s)	Avg travel (s)	Stops / veh	Fuel (L)
Fixed-Time (20s green)	15.45 +/- 0.75	33.0	71.4	1.80	5.56
Max-Pressure, no green-wave	8.11 +/- 0.79	17.7	63.7	1.65	5.28
Max-Pressure + green-wave	9.26 +/- 1.02	21.3	64.8	1.66	5.33

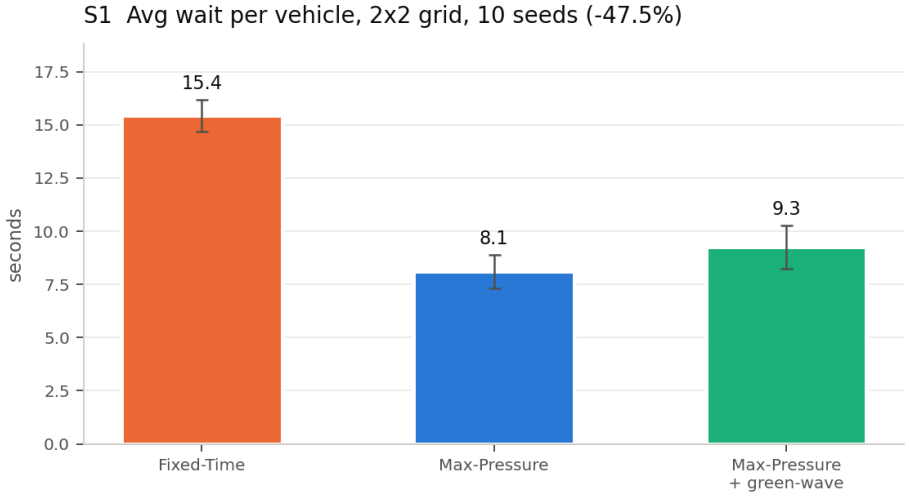


Figure 6. Average wait with standard deviation over 10 seeds

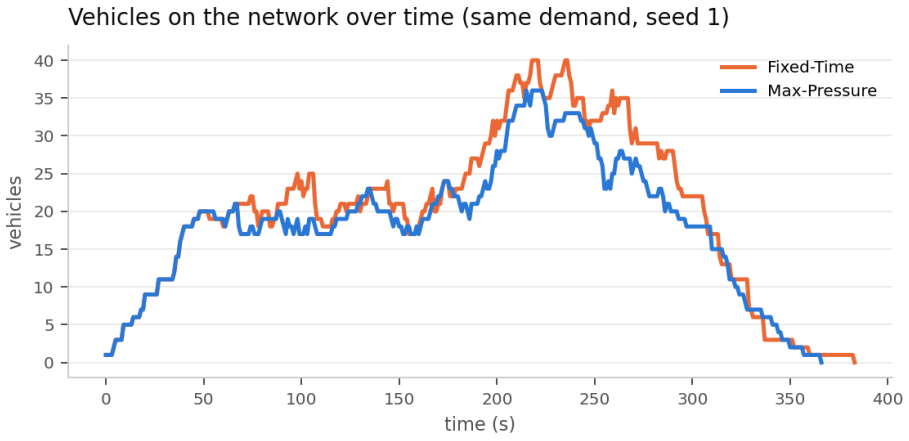


Figure 7. Vehicles on the network over time: Max-Pressure clears the same demand with shorter queues

S2. Rush hour and anti-starvation

Controller	Wait N-S (s)	Wait E-W (s)	Worst E-W wait (s)	E-W cars never served
Fixed-Time	21.88	12.35	25.0	0 / 108
Max-Pressure WITHOUT anti-starvation	2.93	44.27	130.0	14 / 108
Max-Pressure WITH anti-starvation	5.94	12.79	40.0	0 / 108

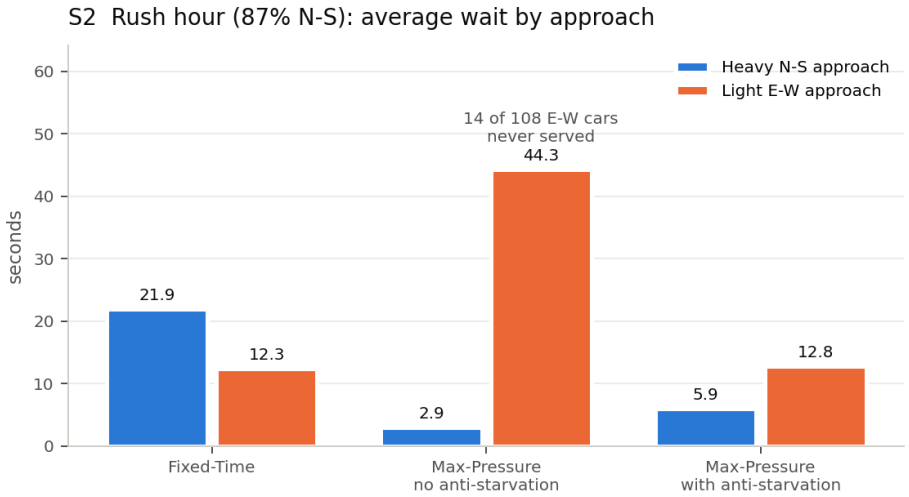


Figure 8. 86.5% of demand on the N-S axis (5 seeds)

S3. Ambulance green corridor

Both runs use the **same Max-Pressure controller and the same background traffic**; only preemption is switched on or off. (The original project compared fixed-time without preemption against Max-Pressure with preemption, which mixed two effects.)

Mode	Ambulance travel (s)	Wait at red (s)	Worst case (s)	Other cars' avg wait (s)
No preemption	54.0	12.2	57.0	5.58
Preemption (MAS)	38.0	0.0	38.0	5.46

Result: **16 s faster (29.6%)**, zero seconds at red lights in all 10 seeds, and no measurable cost to other traffic (-0.12 s).

S4. Accident: static vs local sensing vs TMC broadcast

Policy	Closure: arrived	Slowdown: drove through incident	Slowdown: avg travel (s)
static	0%	12 / 12	144.9
local	100%	12 / 12	144.9
broadcast	100%	0 / 12	95.5

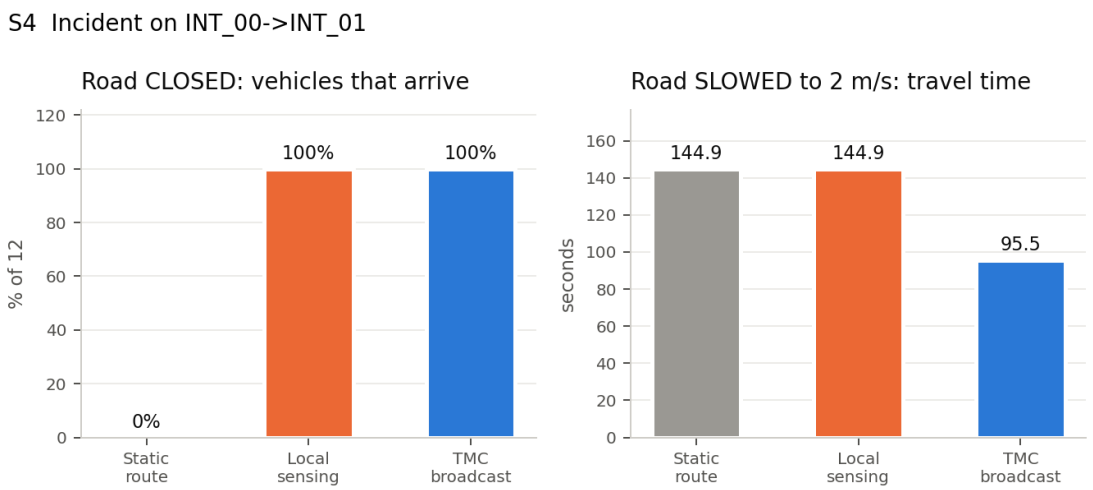


Figure 9. A full closure strands vehicles that never re-plan; for a slowdown only the broadcast helps

With a full closure, the local and broadcast policies give identical results. On a grid, every detour around a single closed road is the same length, and the decision point (INT_00) is the same either way. The broadcast pays off for a slowdown: a vehicle cannot see that a road is slow until it is already driving on it.

S5. Bus Transit Signal Priority

Case	Bus travel (s)	Car wait (s)	Accepted	Rejected	Net passenger-s saved / run
light cross-traffic (0.2 veh/s) TSP OFF	63.8	2.41	0	0	-
light cross-traffic (0.2 veh/s) TSP ON	58.0	4.76	64	0	807
heavy cross-traffic (0.5 veh/s) TSP OFF	63.3	2.87	0	0	-
heavy cross-traffic (0.5 veh/s) TSP ON	59.5	4.12	43	21	453

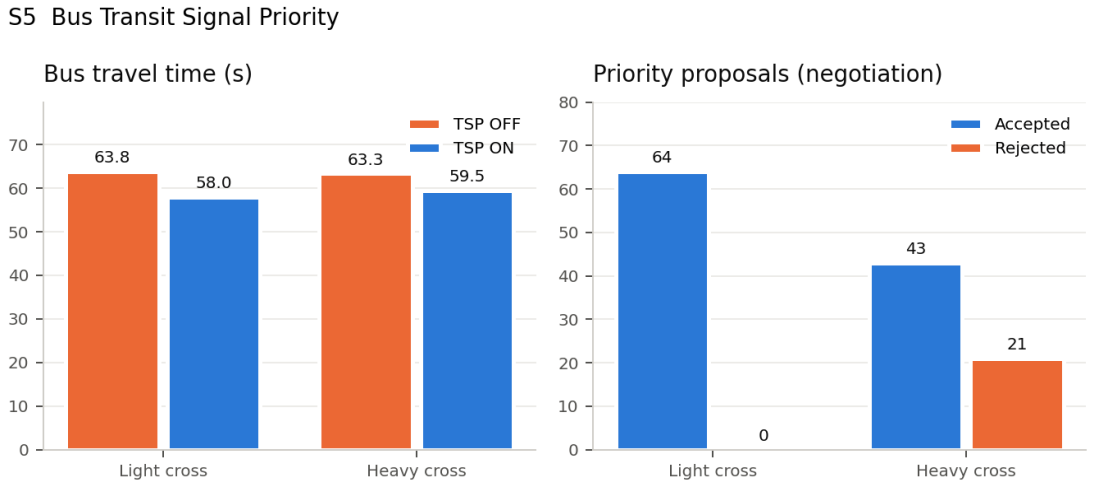


Figure 10. Negotiation in action: under heavy cross traffic a third of the proposals are rejected

Honest trade-off. Bus priority makes each bus about 5.8 s faster, but car waiting on the cross street roughly doubles (2.41 s to 4.76 s). It is still worth it in person-seconds: a bus carries 40

> people, a car about 1.2. Under heavy cross traffic the intersection rejects 21 proposals, which caps the harm.

S6. Hill-climbing tuning

Policy	Training traffic avg wait (s)	Unseen validation traffic avg wait (s)
Fixed-Time	-	17.02
Max-Pressure, default parameters	9.08	8.65
Max-Pressure, tuned by hill climbing	6.85	6.96

Tuned parameters: min green 3 s, max green 15 s, beta 1.5, theta 3. Defaults were 5 s, 30 s, 0.5 and 1.0. The search used 62 simulated evaluations (12.2 s). The improvement carries over to validation seeds that were never used in the search, so it is not over-fitting. Note that the 3 s minimum green sits at the edge of the grid. A real deployment would enforce a pedestrian-safe minimum instead.

S8. Scalability

Grid	Intersections	Vehicles	Agents	Fixed wait (s)	Max-P wait (s)	Reduction	ms / sim-second
2x2	4	90	95	14.1	7.7	45.4%	0.19
3x3	9	143	153	20.7	11.5	44.5%	0.23
4x4	16	190	207	25.7	14.8	42.4%	0.36
5x5	25	234	260	35.7	19.9	44.4%	0.52
6x6	36	296	333	44.6	24.9	44.3%	0.71
8x8	64	396	461	56.8	30.3	46.6%	1.44

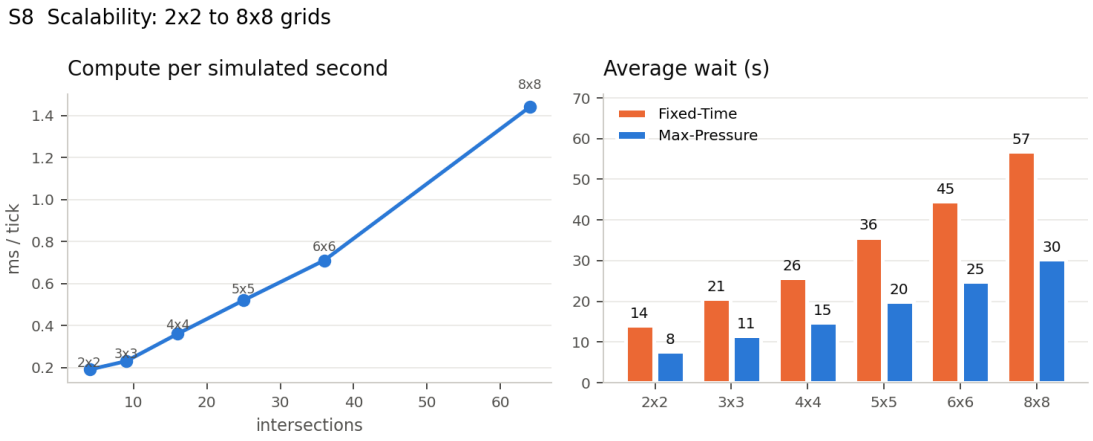


Figure 11. Cost grows linearly with the number of intersections; the benefit holds at every size

7.1 Test summary

43 of 43 automated tests pass (18 s): 23 unit tests, 9 integration tests (agent interaction and safety invariants) and 9 system acceptance tests, one per scenario. The full test cases, expected and actual values, and the defects found during testing are in the separate **Testing Document**.

8. Review 2: Code Structure and Scalability

```
src/
  protocols/message.py        FIPA-ACL message + broker (history, counts)        M5
  agents/base_agent.py        receive -> perceive -> reason -> act                shared
  agents/intersection_agent.py  Max-Pressure, preemption, TSP, green-wave        M2
  agents/vehicle_agent.py     beliefs, A* planning, re-planning, car-following   M3
  agents/emergency_agent.py   look-ahead preemption                              M4
  agents/bus_agent.py         negotiated priority                                M4
  agents/coordinator_agent.py telemetry, incidents, hill-climbing tuner          M5
  algorithms/search_astar.py  BFS, UCS, Greedy, A*, Weighted A*                  M3
  algorithms/max_pressure.py  controller + decision explanation                  M2
  algorithms/hill_climbing.py parameter space, steepest descent, restarts        M5
  environment/                road network (BPR, headway), R x C grids, simulator M1
  demo.py + demo_page.html    the one-page re-route demo                         M1/M3
  visualization/charts.py     report charts                                      M1
  scenarios.py                S1 ... S8
tests/                        43 tests + run_tests.py (JSON report)
```

Component	Cost per decision	Why it scales
Intersection decision	O(1): 8 road counts, 2 scores	Purely local, so adding junctions adds no work to existing ones
A* route plan	O(E log V) worst case; 14 expansions on average at 6x6	Runs inside each vehicle (distributed), only at departure or re-plan
Messages	O(1) point-to-point; O(N) per broadcast	Broadcasts are rare (incidents) and congestion warnings are rate-limited (20 s)
Hill climbing	at most 8 simulations per iteration, memoised	Offline; tuned parameters are pushed once

Measured. Going from 4 to 64 intersections (461 agents), compute time rises from 0.14 to 1.53 ms per simulated second. That is roughly linear and about 650 times faster than real time at 8x8.

9. Limitations and Future Work

9.1 Limitations (stated honestly)

- Two-phase signals with through movements only. Turning movements and pedestrian phases are not modelled.
- Single-lane roads; vehicles queued in front of an ambulance are assumed to pull aside.
- Synthetic Poisson demand on a regular grid, not calibrated to a real city's counts.
- Messages are delivered instantly and never lost; there is no communication delay or failure model.
- The green-wave hand-off helps only on corridors, and bus priority increases cross-street car delay (Section 7).

9.2 Ideas for extension

1. **Reinforcement learning intersections** (Q-learning or DQN on queue states), compared directly against Max-Pressure.
2. **Contract-Net for multiple ambulances** so that conflicting corridor requests are resolved by bidding on urgency.
3. **Pedestrian agents** with push-button requests and a pedestrian-safe minimum green.
4. **Real map import** from OpenStreetMap (osmnx) and calibration against Coimbatore junction counts.
5. **Message loss and delay** to test how robust the protocols are when V2I communication is unreliable.
6. **Simulated annealing or a genetic algorithm** for the coordinator, compared with hill climbing on the same objective.

10. Conclusion

A decentralised multi-agent design meets every objective. Local Max-Pressure control with an anti-starvation term cuts waiting by 47.5% and never leaves a car unserved. A* over each vehicle's own beliefs gives optimal routes at a fraction of UCS's search effort. Explicit protocols let an ambulance cross without stopping, let a bus negotiate rather than override, and let one broadcast about an incident save vehicles about a third of their travel time. The coordinator improves every controller by hill climbing without taking control away from them. Because every claim is tied to a regenerable result and an automated test, the system can be checked rather than taken on trust.

References

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021 (agents, PEAS, A*, local search).

[2] P. Varaiya, "Max pressure control of a network of signalized intersections," Transportation Research Part C, vol. 36, pp. 177-195, 2013.

[3] Bureau of Public Roads, Traffic Assignment Manual, U.S. Dept. of Commerce, 1964 (BPR travel-time function).

[4] Transportation Research Board, Highway Capacity Manual, 7th ed., 2022 (saturation flow, 1800 veh/h/lane).

[5] FIPA, "FIPA ACL Message Structure Specification (SC00061G)" and "FIPA Request / Propose Interaction Protocols," 2002.

[6] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. SSC, vol. 4, no. 2, 1968.

Appendix A. Viva Question Bank (by owner)

M1 - Environment & Simulation

Question	Answer
Why a 2 s discharge headway?	It is the Highway Capacity Manual saturation flow of 1800 vehicles per hour per lane. Without it, a whole queue could cross in one second and signal control would barely matter.
Why is the environment called continuous if you use 1 s ticks?	Positions and speeds are real-valued. Only time is discretised, for tractability, and 1 s is finer than any signal decision.
How do you know results are not luck?	Runs are seeded (IT-09 checks determinism), averaged over 5-10 seeds with standard deviations, and every vehicle runs to completion.

M2 - Intersection Agent

Question	Answer
Why Max-Pressure instead of 'longest queue first'?	It subtracts the downstream queue, so it will not push cars into a full road (spillback). Varaiya proved it stabilises any demand that can be stabilised.
What happens without the starvation term?	In S2, 14 side-street cars were never served. One car against zero never beats the switching threshold.
Can preemption cause a crash?	No. Every switch, including an emergency one, goes through a 3 s yellow. IT-02 checks over 400 s with ambulances and buses that no green ever changes directly to the other green.
Why does the green-wave not help on the grid?	Random origin-destination pairs create no platoons, so extending a green for predicted arrivals delays the other axis. It helps on a corridor (-32%).

M3 - Vehicle & Search

Question	Answer
Is your heuristic admissible under congestion?	Yes. Congestion only increases true cost, and h uses the fastest possible speed and the straight-line distance, so h stays a lower bound (UT-01).
Why not Greedy? It expands fewer nodes.	It is optimal in only 51.5% of trips. A* costs a few more expansions and is optimal every time.
What does the vehicle know?	Only its beliefs: roads it was told about or saw itself. That is why the broadcast matters for slowdowns (S4).
What if the destination becomes unreachable?	Search returns failure (UT-07). The vehicle keeps its route and waits, and re-plans when a ROAD_CLEARED broadcast arrives.

M4 - Priority Vehicles

Question	Answer
Why is the ambulance's preemption not a pure override?	It still respects the yellow clearance, and it releases the junction with CANCEL so Max-Pressure resumes immediately (IT-01).
Why can a bus be refused?	Priority for 40 passengers should not create a long queue of cars. The intersection refuses when the cross queue is over 6 or an ambulance is active (IT-03, IT-04).
Does preemption hurt other traffic?	Measured change in other cars' average wait: -0.12 s, which is negligible.

M5 - Coordinator & Testing

Question	Answer
Why hill climbing and not exhaustive	2,058 settings x 3 simulations is about 6,000 runs. Hill climbing needed 62, and

Question	Answer
search?	restarts reduce the risk of a poor local optimum.
How do you know tuning did not over-fit?	It was evaluated on 5 validation seeds never seen during search: 8.65 s to 6.96 s.
Is the coordinator a single point of failure?	No. Signals keep running Max-Pressure locally if it disappears. It only adds information (alerts) and tuning.
What did testing find?	Five defects in the original code, including double reasoning per tick, ignored departure times and an unfair ambulance baseline. See the Testing Document.